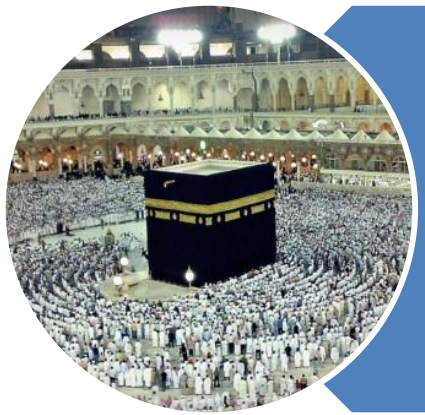
Data Structures - I

CPCS 204

رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Trees

Introduction

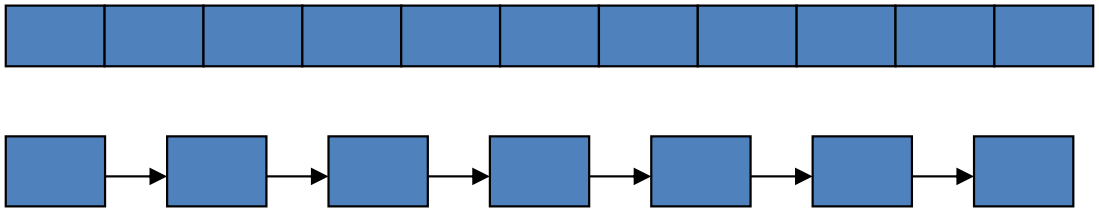
Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

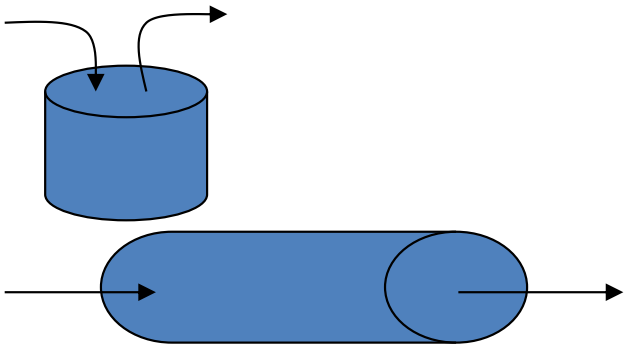
CPCS 204

Trees - Introduction

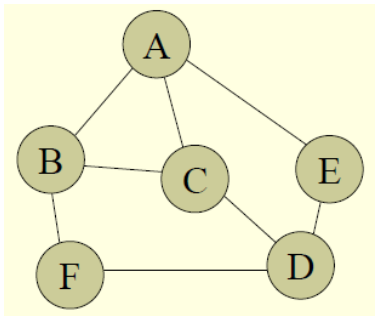
- Arrays
- Linked List

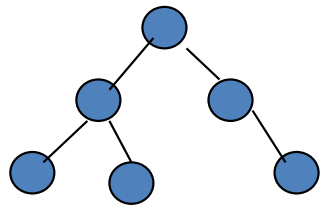


- Stacks
- Queues



- **Tree**
- Graph



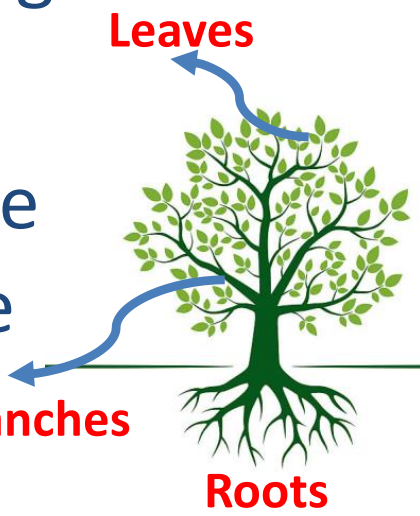


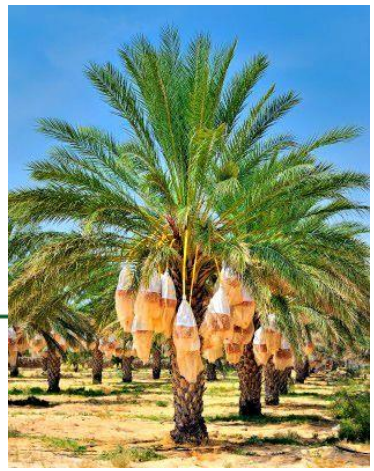
Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Trees - Introduction

- Definition
  - A **tree** is a **hierarchical** data structure that organizes data elements, called **nodes**, by connecting them with links, called **branches**
- Examples
  - Dates Tree
  - Palm Tree
- Tree has





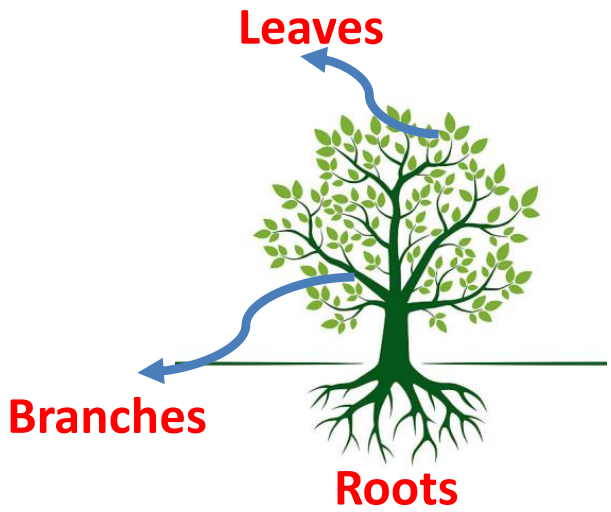
Dr. Jonathan (Yahya) Cazalas

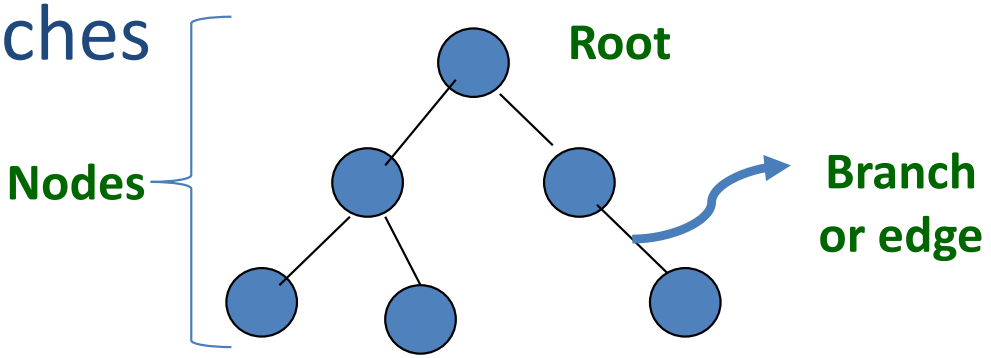
Dr. Muhammad Umair Ramzan

CPCS 204

Trees - Introduction

- Tree in nature consists of
  - Roots
  - Branches
  - Leaves
- Tree in Computing
  - Edge or Branches
  - Nodes





Dr. Jonathan (Yahya) Cazalas

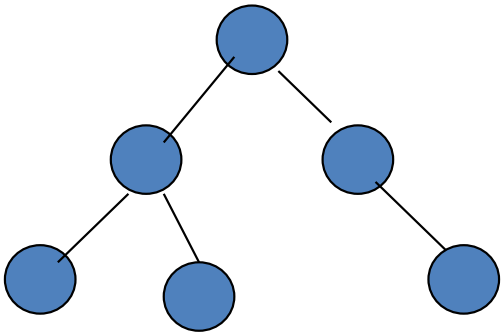
Dr. Muhammad Umair Ramzan

Trees - Introduction

- Array's members
  - Elements
    - Element of array

- Linked List's members
  - Nodes
    - Node of a linked List

- Tree's members
  - **Nodes**
    - Node of a tree

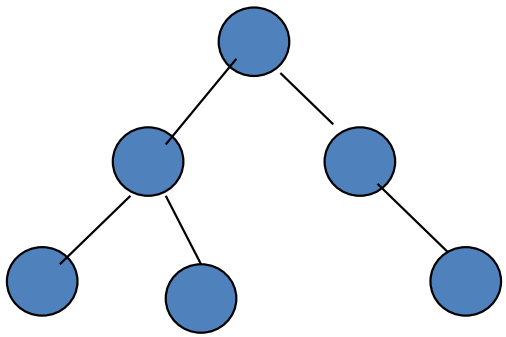


Trees - Introduction

- **Relationship**
  - Arrays

    - Predecessor - Successor
  - Linked List

    - Predecessor - Successor
  - Tree
    - **Parent - Child**



Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

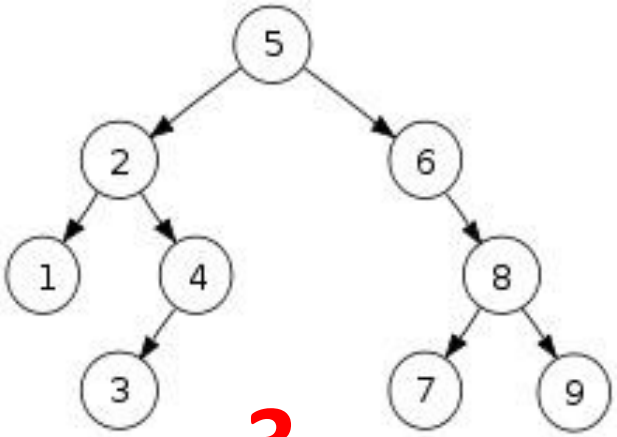
CPCS 204

Trees - Introduction

- **Size Measurement**
  - Arrays

    - Length of array
    - # of elements i.e. **11**
  - Linked List

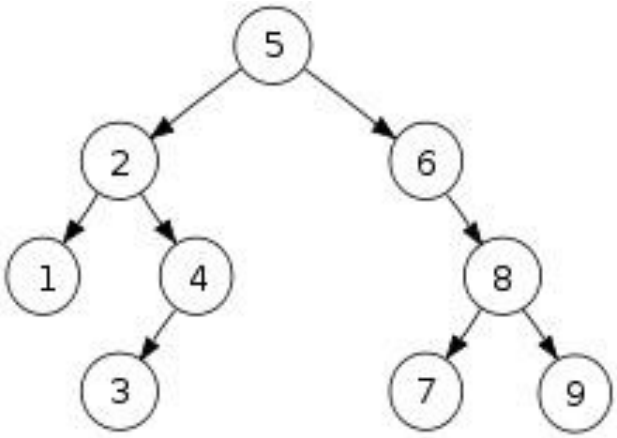
    - Length of linked list
    - # of nodes i.e. **7**
  - Tree
    - **Height** or Depth of Tree
    - # of **edges** along **longest path** from the root to a leaf in that tree
    - Tree with one node has height **0**



3

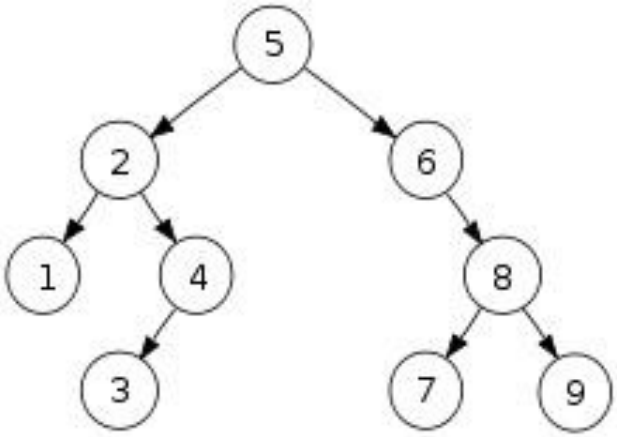
CPCS 204

Trees - Introduction



- **Types of Node**
  - **Child** Node
    - A node in a tree can have several successors, which we refer to as children
    - **2** & **6** are the children of 5
  - **Parent** Node
    - A node's predecessor would be its parent
    - **5** is parent of 2 and 6 **& 8** is parent of 7 and 9

Trees - Introduction



- **Node Types (Cont.)**
  - **Root** Node
    - A node which has no parent
    - **5** is the root node
  - **Leaf** Node
    - A node which has no child
    - **1, 3, 7, 9** are leaf nodes
  - **Interior** Node
    - A node which is neither a *root* nor a *leaf*
    - **2, 4, 6, 8** are interior nodes

CPCS 204

Trees - Summary

- Abstract Data Type (ADT)
- Hierarchical Data Structure
- Nodes
  - Root, Parent, Child, Leaf, Interior
- Parent – Child relationship
- Size is measured in terms of height
  - # of edges from root to leaf along the longest path
- Logical Data Structure
- Implementation
  - Array
  - Linked List

Trees

Types of Tree

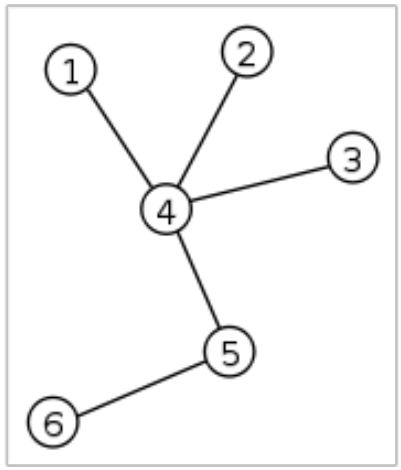
Dr. Jonathan (Yahya) Cazalas

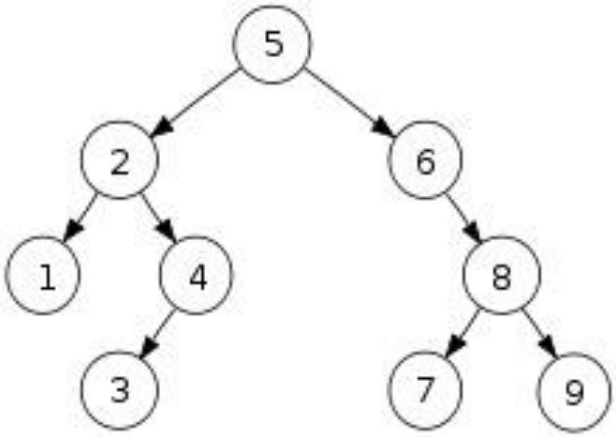
Dr. Muhammad Umair Ramzan

CPCS 204

Trees - Introduction

- **Types of Tree**
  - General Tree
    - Any number of roots
    - Any number of children per node
  - Binary Tree
    - Only **one root**
    - Maximum **2 children** per node





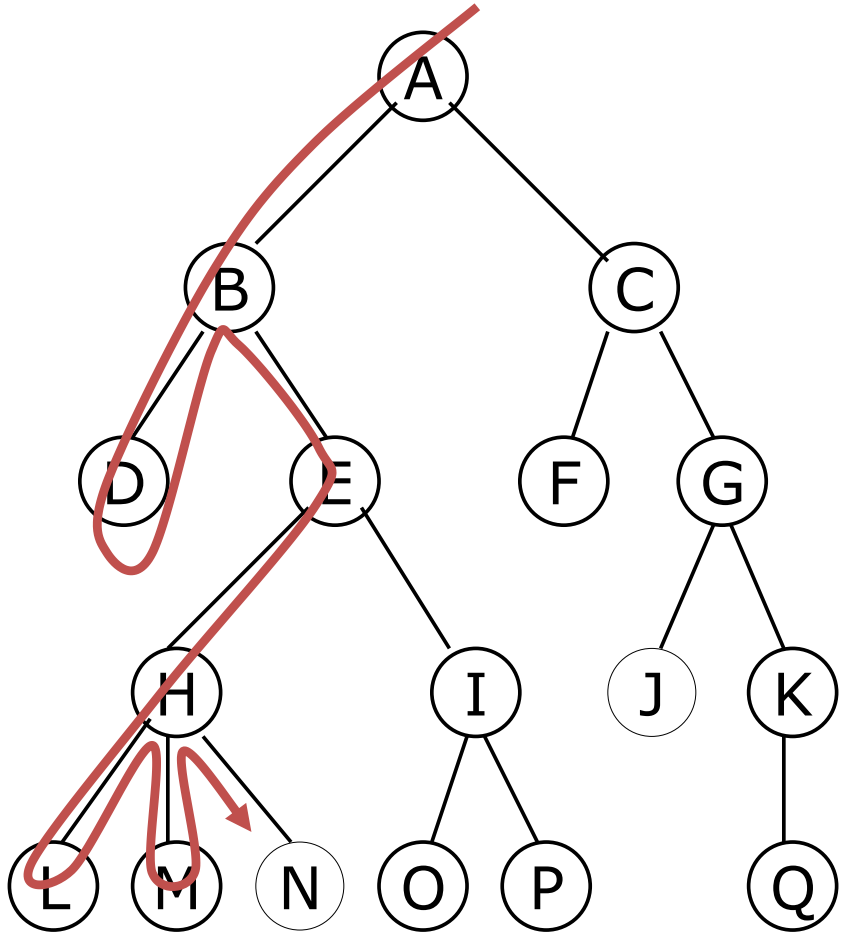
General Trees - Introduction

- **Tree Traversal**
  - Depth First Search (DFS)
    - It explores a path **all the way to a leaf** before backtracking and exploring another path
  - Breadth First Search (BFS)
    - It explores **all nodes nearest the root** before exploring nodes further away

CPCS 204

General Trees - Introduction

- **Depth First Search (DFS)**
  - It explores a path **all the way to a leaf** before backtracking and exploring another path



A B D E H L M N I O P C F G J K Q

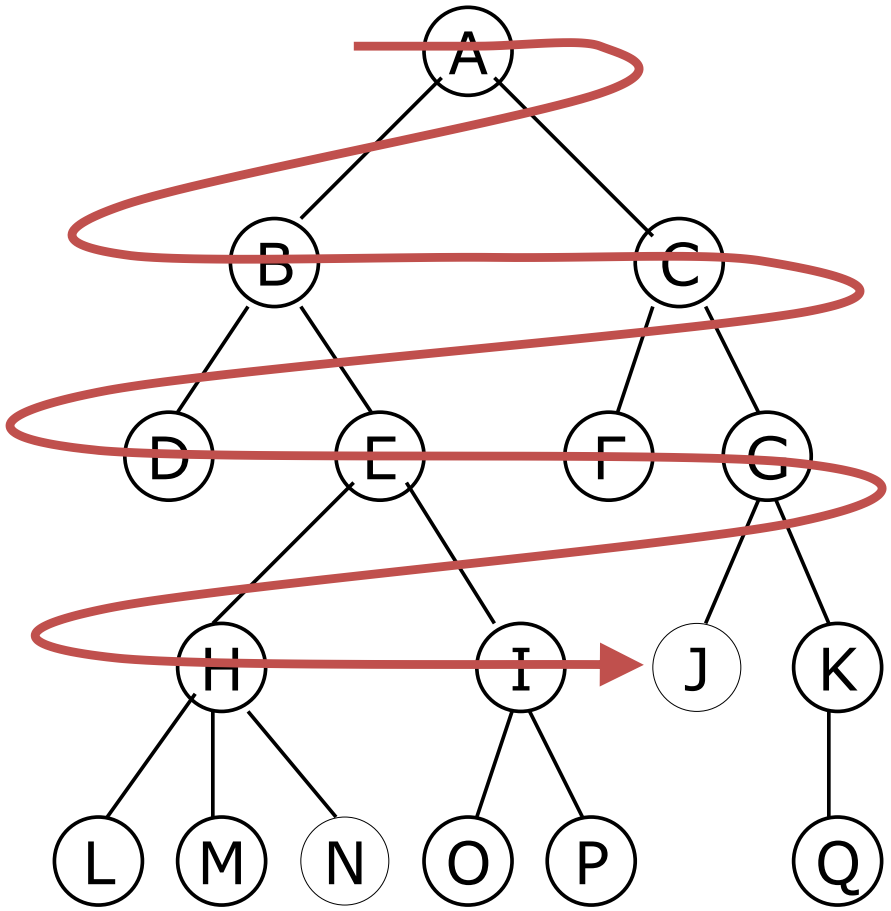
Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

General Trees - Introduction

- **Breadth First Search (BFS)**
  - It explores **all nodes nearest the root** before exploring nodes further away



A B C D E F G H I J K L M N O P Q

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Binary Trees

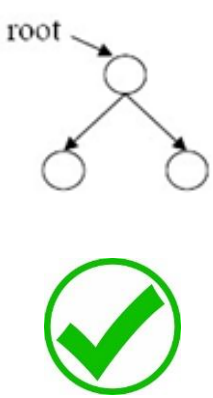
Introduction

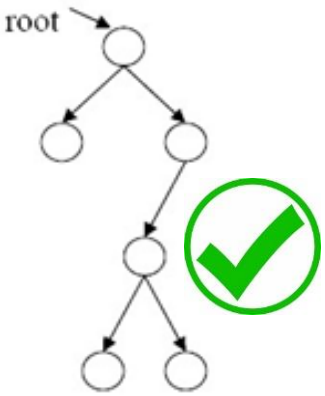
Dr. Jonathan (Yahya) Cazalas

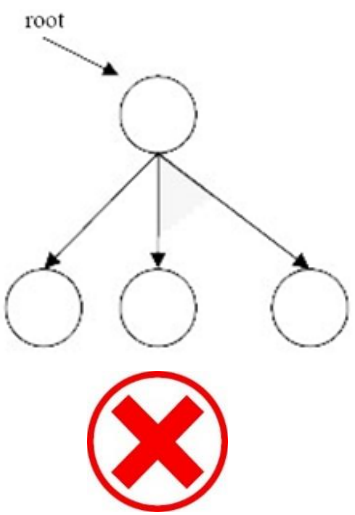
Dr. Muhammad Umair Ramzan

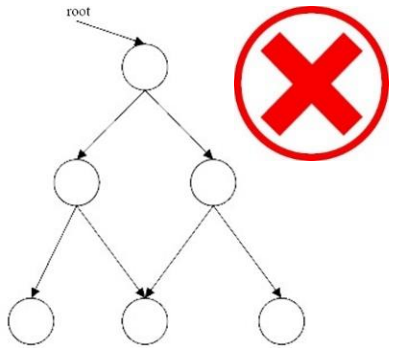
Binary Tree - **Introduction**

- **Definition**
  - A tree in which each node can have a maximum of two children
    - Each node can have no child, one child, or two children
    - And a child can only have one parent



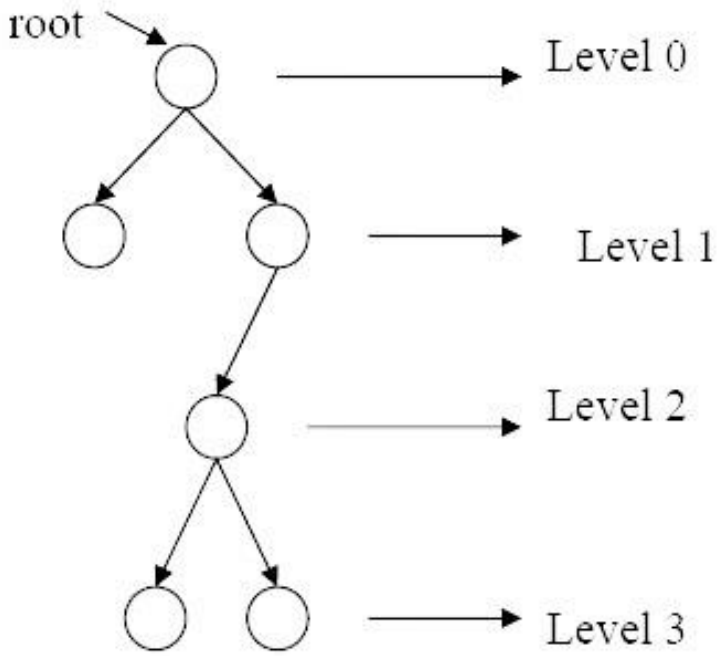






Binary Tree - **Introduction**

- **Level**
  - Level of a node is the *number of edges* between the node and the root
  - Root is at *level 0*

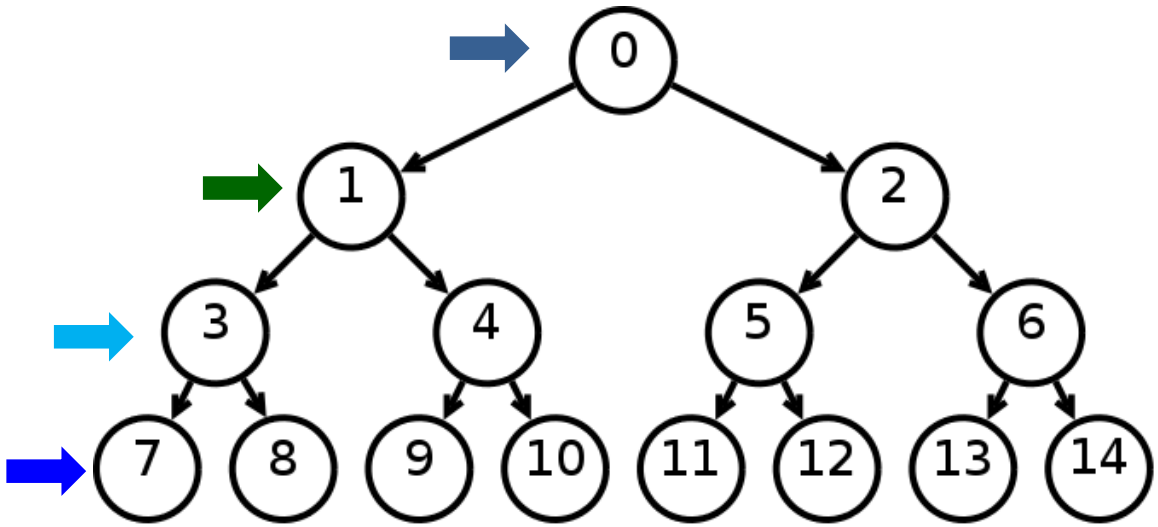


Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Binary Tree - Introduction

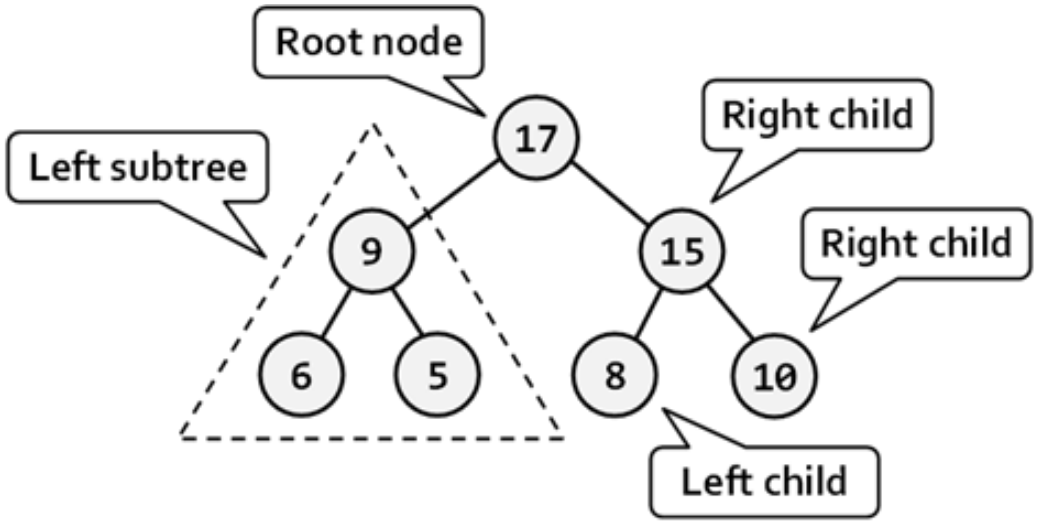
- **Maximum nodes at any level**
  - Level "i"
  - **$N = 2^i$**
    - i=0
    - $N = 2^0 = 1$
    - i=1
    - $N = 2^1 = 2$
    - i=2
    - $N = 2^2 = 4$
    - i=3
    - $N = 2^3 = 8$



CPCS 204

Binary Tree - **Introduction**

- **Sub Tree**
  - A **Sub Tree** of a tree T, is a tree consisting of **a node** in T, and all of **its children** (descendants).



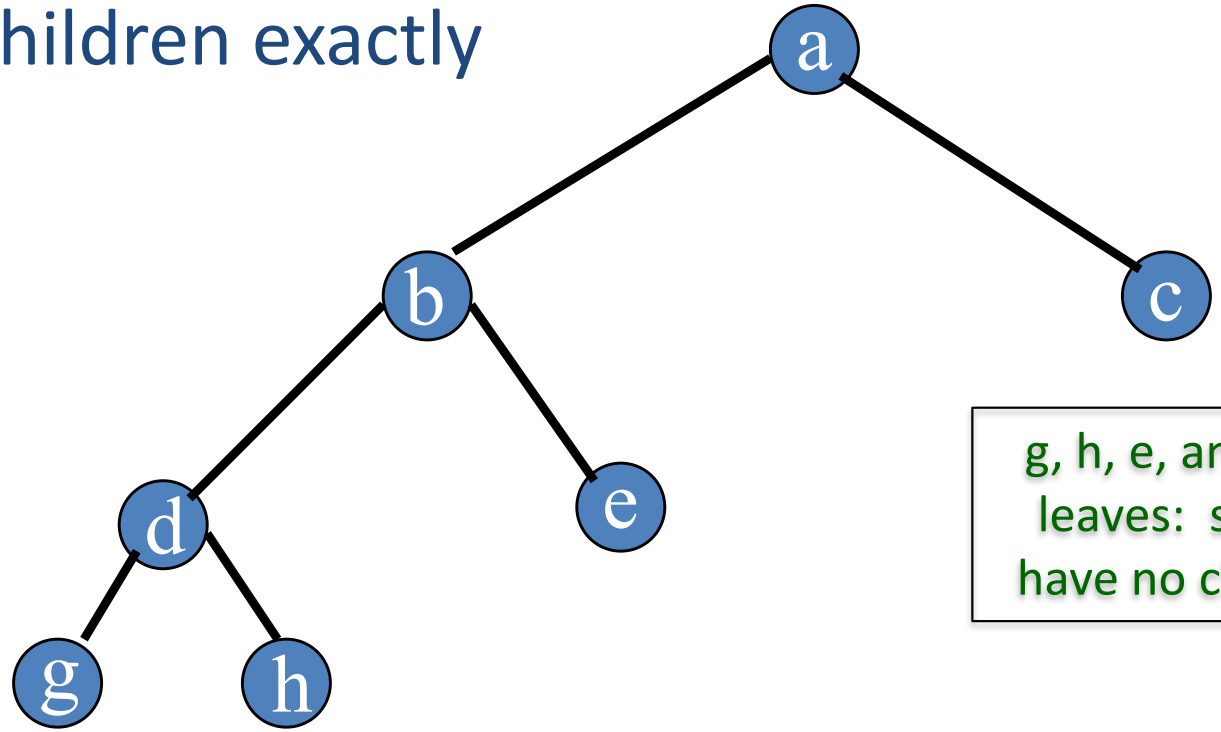
- Left Subtree
- Right Subtree

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Binary Tree - **Introduction**

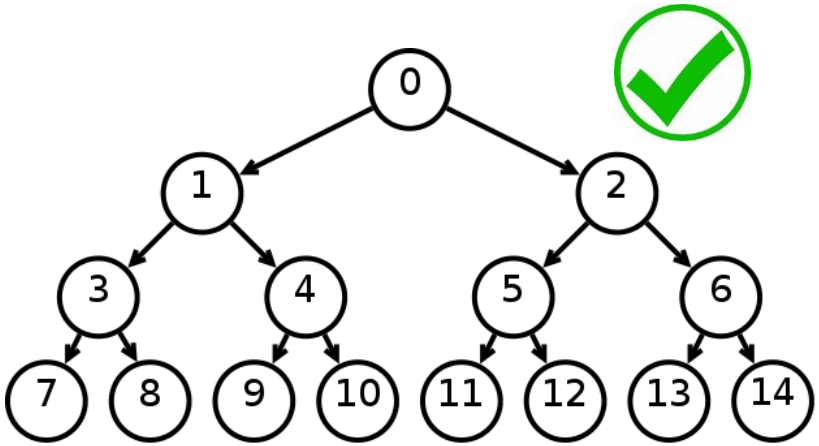
- **Full Binary Tree**
  - Every node, other than the leaves, **has two** children exactly

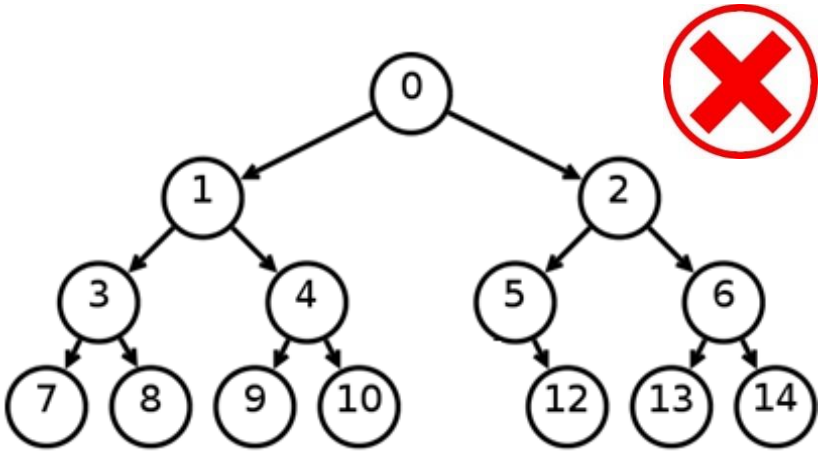


g, h, e, and c are leaves: so they have no children.

Binary Tree - **Introduction**

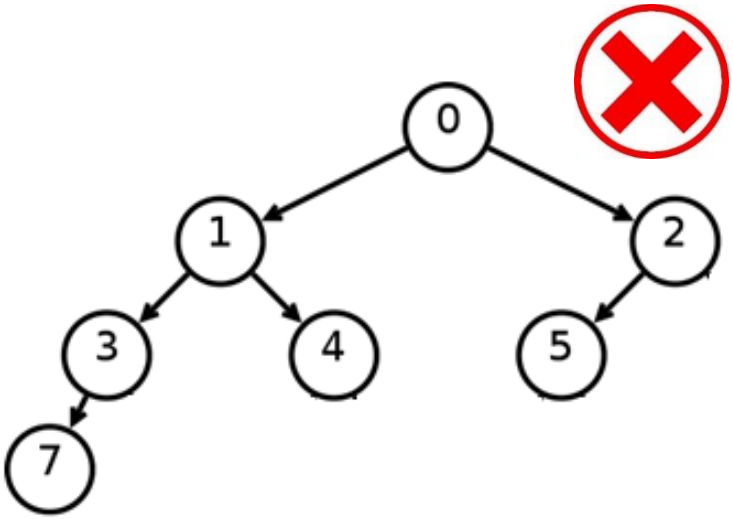
- **Complete Binary Tree**
  - Every level, except possibly the last, is completely filled, and all nodes are as far left as possible.

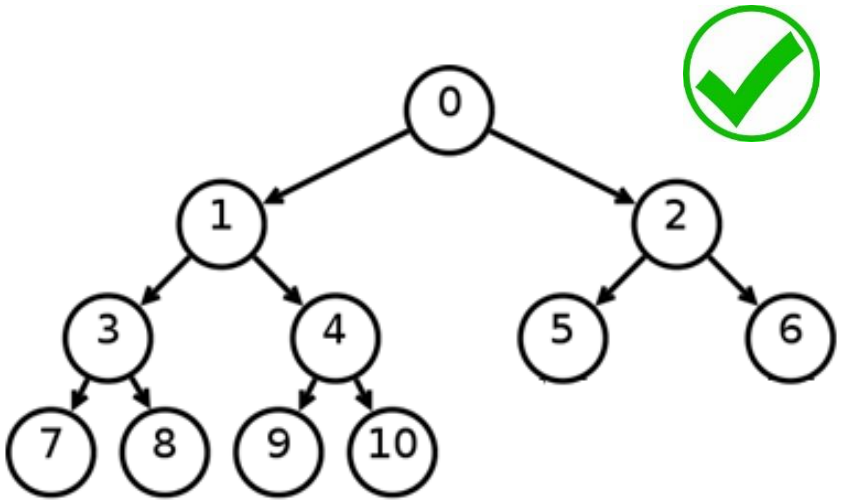




Binary Tree - **Introduction**

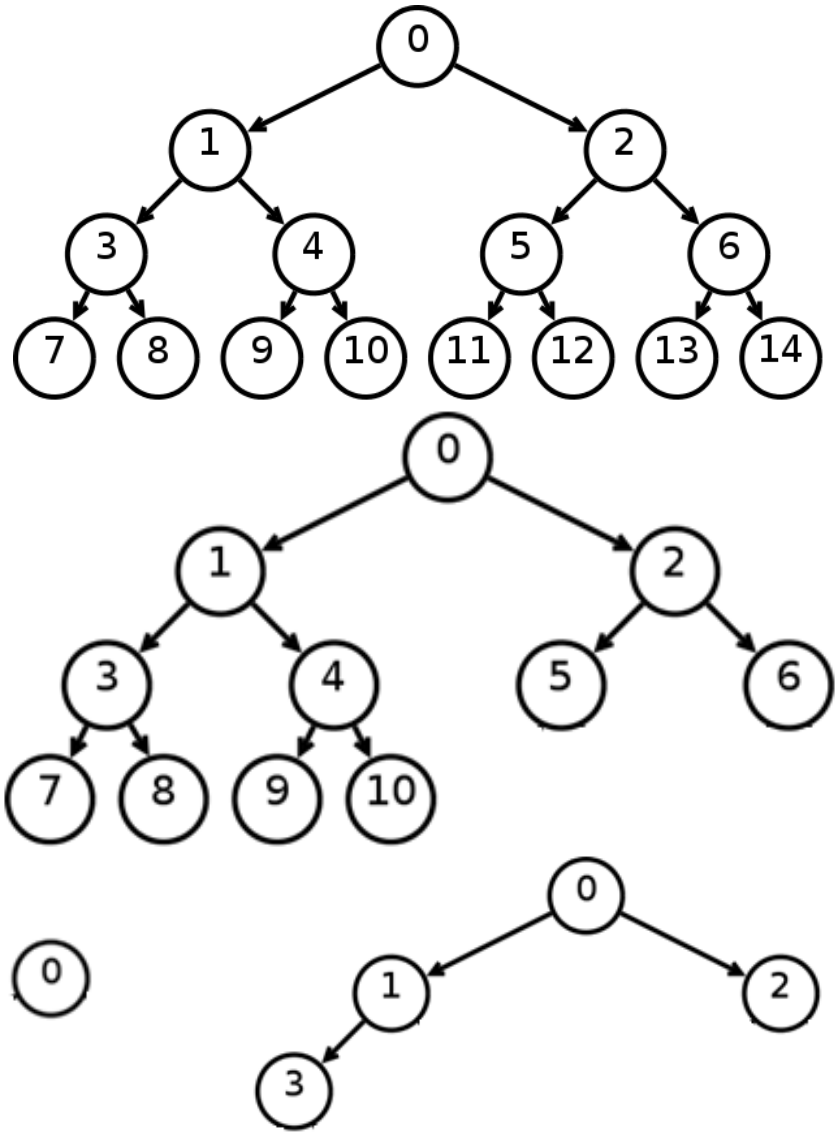
- **Complete Binary Tree**
  - Every level, except possibly the last, is completely filled, and all nodes are as far left as possible.





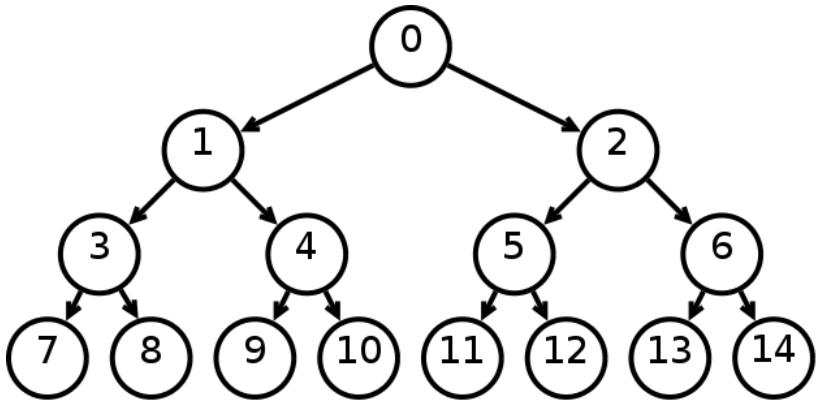
Binary Tree - Introduction

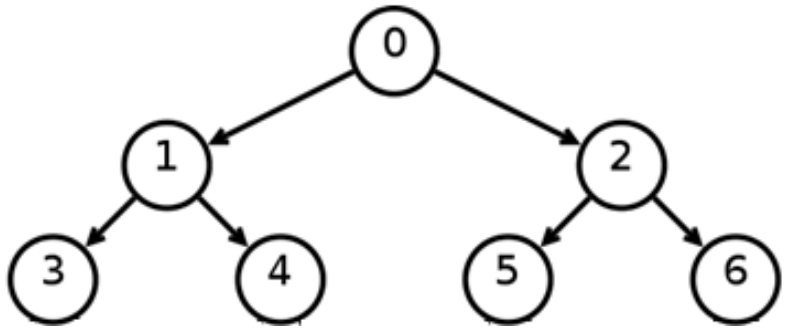
- Complete Binary Tree
- Height $h = \lfloor \log_2 N \rfloor$
  - h = floor($\log_2$ 15)
  - h = floor(3.906) = 3
  - h = floor($\log_2$ 11)
  - h = floor(3.459) = 3
  - h = floor($\log_2$ 4)
  - h = floor(2) = 2
  - h = floor($\log_2$ 1)
  - h = floor(0) = 0

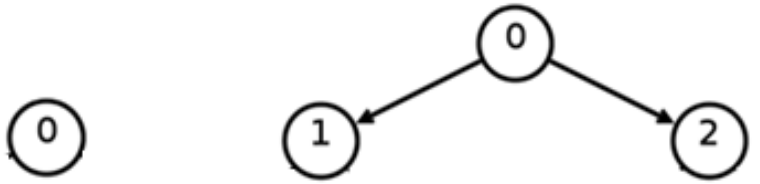


Binary Tree - Introduction

- Complete Binary Tree
- Max. no. of nodes
  - $N = 2^{h+1} - 1$
    - $N = 2^{3+1} - 1 = 2^4 - 1$
    - $= 16 - 1 = 15$
    - $N = 2^{2+1} - 1 = 2^3 - 1$
    - $= 8 - 1 = 7$
    - $N = 2^{1+1} - 1 = 2^2 - 1$
    - $= 4 - 1 = 3$
    - $N = 2^{0+1} - 1 = 2^1 - 1$
    - $= 2 - 1 = 1$







Binary Trees

Traversal

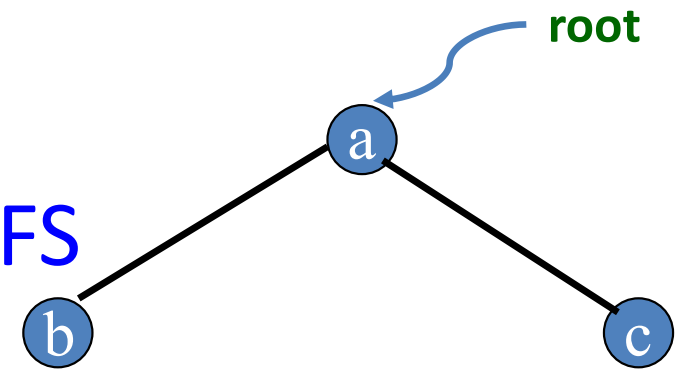
Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Binary Tree - **Introduction**

- **Binary Tree Traversal**
- It is designed Based on DFS
  - **3 - variants**
    - **Pre Order**
      - **Root – Left - Right**
    - **In Order**
      - **Left – Root - Right**
    - **Post Order**
      - **Left – Right - Root**



Pre Order:

a b c

In Order:

b a c

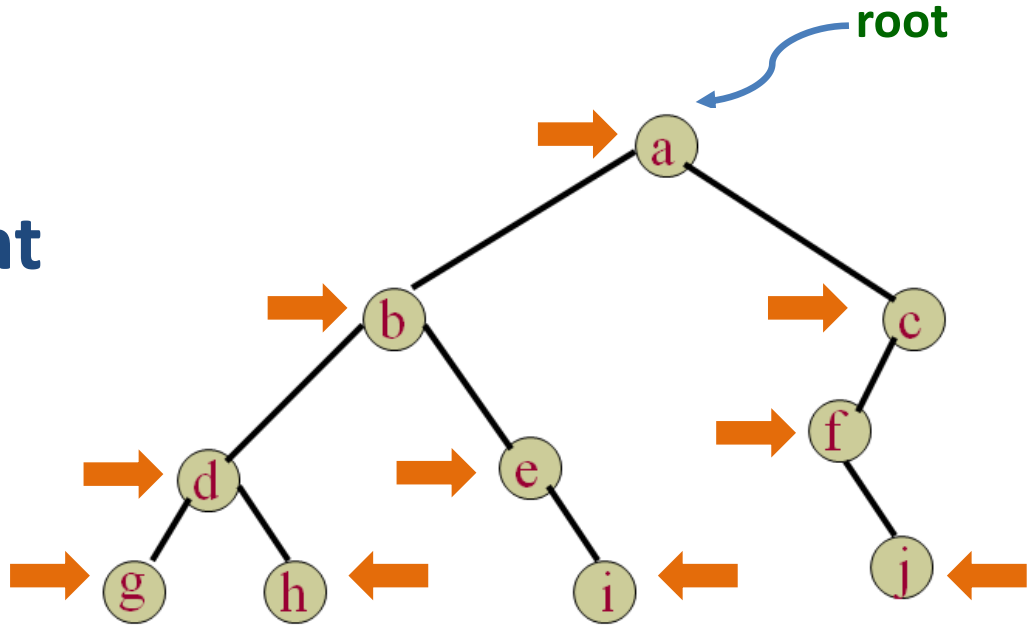
Post Order:

b c a

CPCS 204

Binary Tree Traversal – Pre Order

- **Pre Order**
  - **Root – Left - Right**



Traversal: a b d g h e i c f j

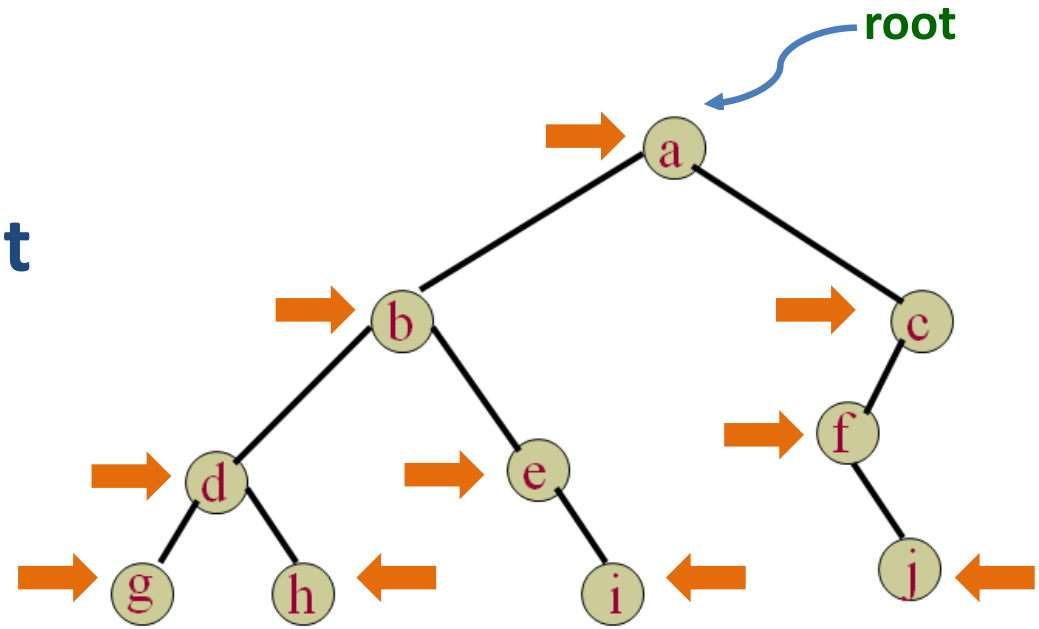
Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Binary Tree Traversal – In Order

- **In Order**
  - **Left - Root –Right**



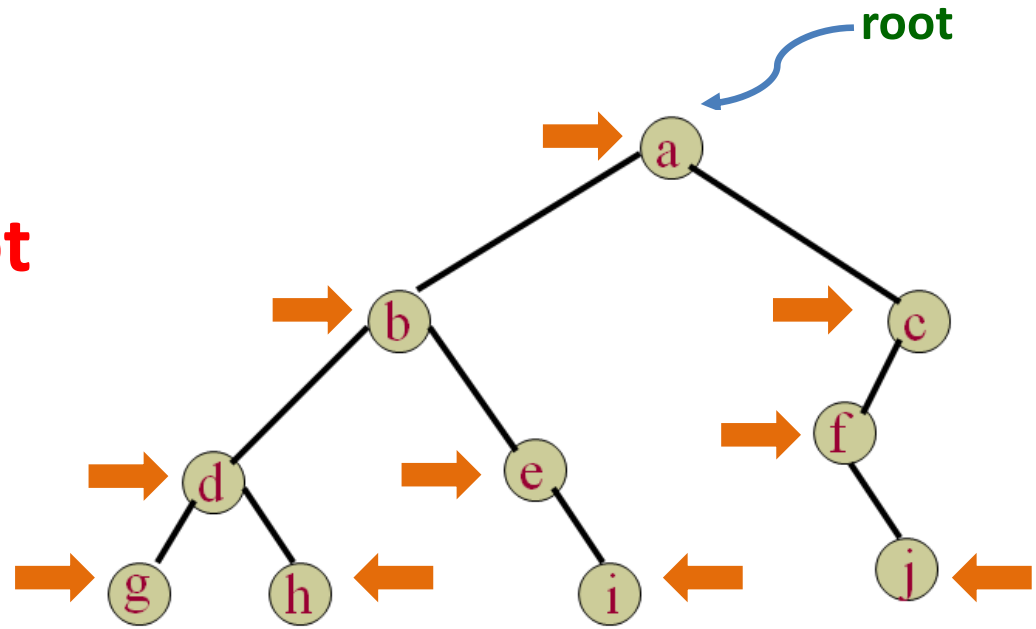
Traversal: g d h b e i a f j c

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Binary Tree Traversal – Post Order

- **Post Order**
  - **Left - Right - Root**



Traversal: g h d i e b j f c a

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Binary Trees

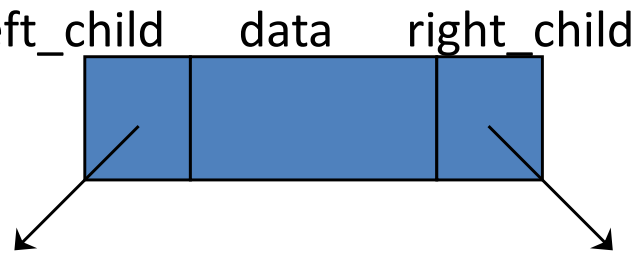
Implementation

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Binary Tree - **Implementation**

- A binary tree has a natural implementation using **linked storage** i.e. Linked List
- Each node of a binary tree has both **left** and **right** subtrees that can be reached with pointers.
- **Double Linked List**



Binary Tree – Node Class

```
public class BTnode {
   public int data;
   public BTnode left;
   public BTnode right;

    public BTnode (int i) {
       this(i, null, null);
    }

    public BTnode (int i, BTnode n, BTnode p) {
       data  = i;
       left  = n;
       right = p;
    }
}
```

Class – BinaryTree

```
public class BinaryTree {
    private BTnode root;

    // Constructor
    public BinaryTree() {

            root = null;
    }


    // All methods will be written here
}
```


Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Binary Tree Implementation

Traversal

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Binary Tree – Pre Order

- **Root – Left – Right**
- the root is visited before its left and right sub trees

```
public void preorder (BTnode p) {
    if (p != NULL) {
        System.out.print(p.data + " ");
        preorder(p.left);
        preorder(p.right);
    }
}
```

Binary Tree – In Order

- **Left - Root –Right**
- the root is visited between the left and right sub trees

```
public void inorder (BTnode p) {
    if (p != NULL) {
        inorder(p.left);
        System.out.print(p.data + " ");
        inorder(p.right);
    }
}
```

Binary Tree – Post Order

- **Left - Right - Root**
- the root is visited after both the left and right sub trees

```
public void postorder (BTnode p) {
    if (p != NULL) {
        postorder(p.left);
        postorder(p.right);
        System.out.print(p.data + " ");
    }
}
```

CPCS 204

Binary Tree Implementation

Searching

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Binary Tree - Searching

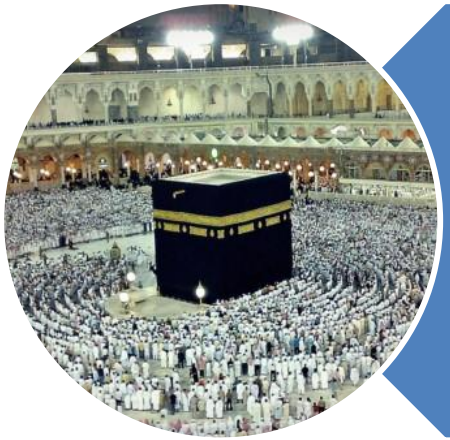
- Searching for a node in an arbitrary tree
- If found return "true" otherwise "false"
- Searching will take O(n) time

```
public boolean searchTree( BTnode p, int data ) {
   if (p == null) {
      return false;
   }

   if (data == p.data) {
      return true;
   }

   return  searchTree(p.left, data) || searchTree(p.right, data);
}
```

Data Structures - I

"No Bonus" marks for this course anymore

CPCS 204



Allah (Alone) is Sufficient for us, and He is the Best Disposer of affairs (for us).(Quran 3 : 173)

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan